Titanic Dataset Exploratory Data Analysis & Cleaning

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Load dataset

In [19]:
```
df = pd.read_csv(r"C:\Users\HP\Desktop\data analysis portfolio\Elevvo Internship\tr
print(df.shape)
df.head()
```

```
(891, 12)
```

Out[19]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500

Handle missing values and convert types

Impute Age with median per (Pclass, Sex)

In [23]:
```
age_medians = df.groupby(['Pclass','Sex'])['Age'].median()
df['Age'] = df.apply(lambda row: age_medians.loc[row['Pclass'], row['Sex']] if pd.i
else row['Age'], axis=1)
```

Fill Embarked with mode

In [31]:
```
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

Fill Fare with median if missing

In [33]:
```
df['Fare'].fillna(df['Fare'].median())
```

Out[33]:
```
0       7.2500
1      71.2833
2       7.9250
3      53.1000
4       8.0500
        ...
886    13.0000
887    30.0000
888    23.4500
889    30.0000
890     7.7500
Name: Fare, Length: 891, dtype: float64
```

Convert columns to categorical

In [36]:
```
for col in ['Pclass','Survived','Sex','Embarked']:
 df[col] = df[col].astype('category')
```

In [38]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    category
 2   Pclass       891 non-null    category
 3   Name         891 non-null    object
 4   Sex          891 non-null    category
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     891 non-null    category
dtypes: category(4), float64(2), int64(3), object(3)
memory usage: 59.8+ KB
```

Summary statistics

In [41]:
```
print("\nNumeric summary:")
display(df.describe(include=[np.number]).T)
```

```
Numeric summary:
```

	count	mean	std	min	25%	50%	75%	max
PassengerId	891.0	446.000000	257.353842	1.00	223.5000	446.0000	668.5	891.0000
Age	891.0	29.112424	13.304424	0.42	21.5000	26.0000	36.0	80.0000
SibSp	891.0	0.523008	1.102743	0.00	0.0000	0.0000	1.0	8.0000
Parch	891.0	0.381594	0.806057	0.00	0.0000	0.0000	0.0	6.0000
Fare	891.0	32.204208	49.693429	0.00	7.9104	14.4542	31.0	512.3292

In [43]:
```
print("\nCategorical summary:")
display(df.describe(include=['category','object']).T)
```

```
Categorical summary:
```

	count	unique	top	freq
Survived	891	2	0	549
Pclass	891	3	3	491
Name	891	891	Braund, Mr. Owen Harris	1
Sex	891	2	male	577
Ticket	891	681	347082	7
Cabin	204	147	B96 B98	4
Embarked	891	3	S	646

Group-based survival insights

In [50]:
```
# Convert Survived to numeric if it's categorical or object
df['Survived'] = pd.to_numeric(df['Survived'], errors='coerce')

# Now group
surv_by_sex = df.groupby('Sex', observed=True)['Survived'].mean()
surv_by_pclass = df.groupby('Pclass', observed=True)['Survived'].mean()
surv_by_embarked = df.groupby('Embarked', observed=True)['Survived'].mean()
```

In [52]:
```
print("Overall survival rate:", df['Survived'].mean())
print("\nSurvival by Sex:")
print(surv_by_sex)
print("\nSurvival by Pclass:")
print(surv_by_pclass)
```

```
print("\nSurvival by Embarked:")
print(surv_by_embarked)
```

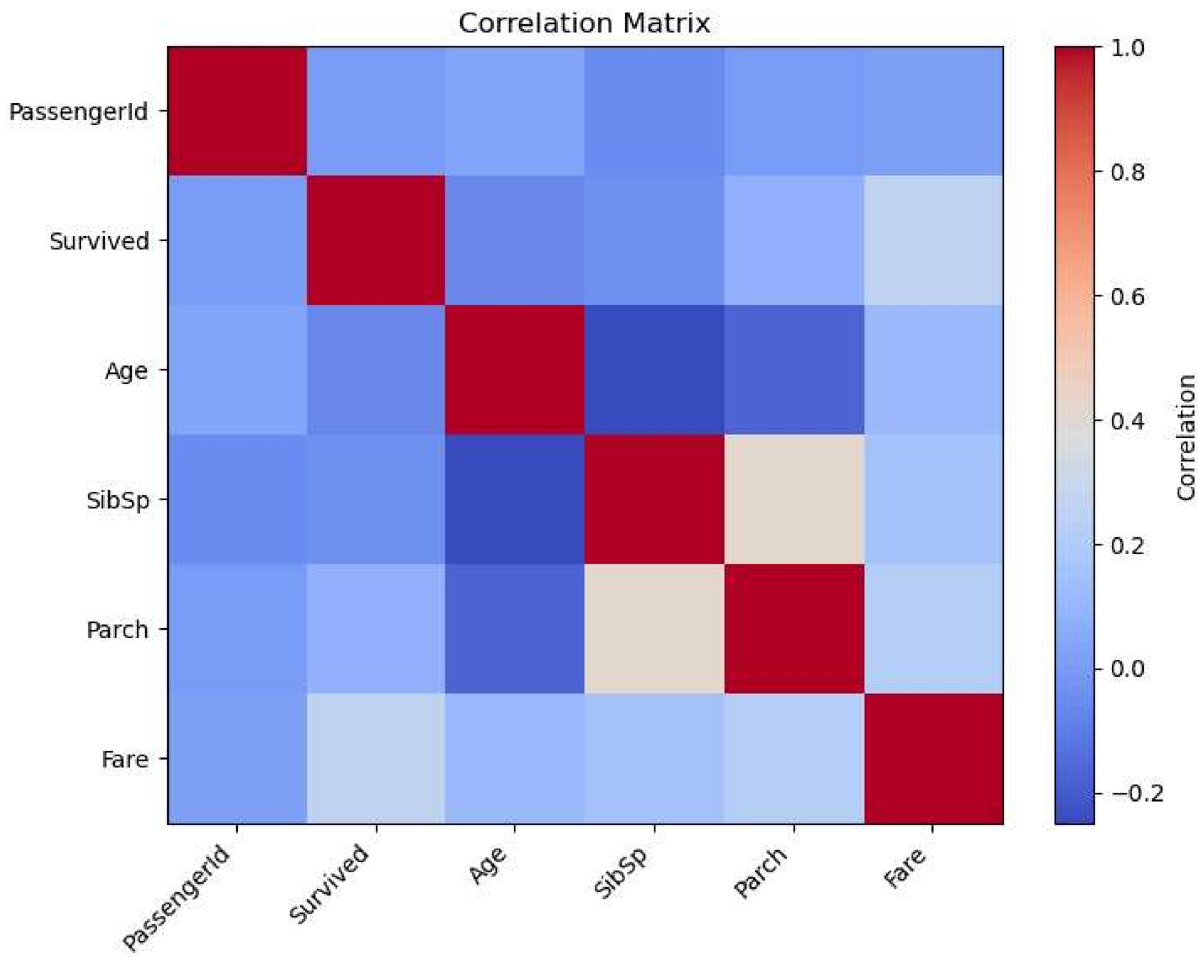
```
Overall survival rate: 0.3838383838383838

Survival by Sex:
Sex
female    0.742038
male      0.188908
Name: Survived, dtype: float64

Survival by Pclass:
Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64

Survival by Embarked:
Embarked
C    0.553571
Q    0.389610
S    0.339009
Name: Survived, dtype: float64
```

Exploratory Data Analysis

Correlation matrix for numeric features

In [56]:

```
corr = df.select_dtypes(include=[np.number]).corr()
plt.figure(figsize=(8,6))
plt.imshow(corr, aspect='auto', cmap='coolwarm')
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha='right')
plt.yticks(range(len(corr.columns)), corr.columns)
plt.colorbar(label='Correlation')
plt.title('Correlation Matrix')
plt.show()
```

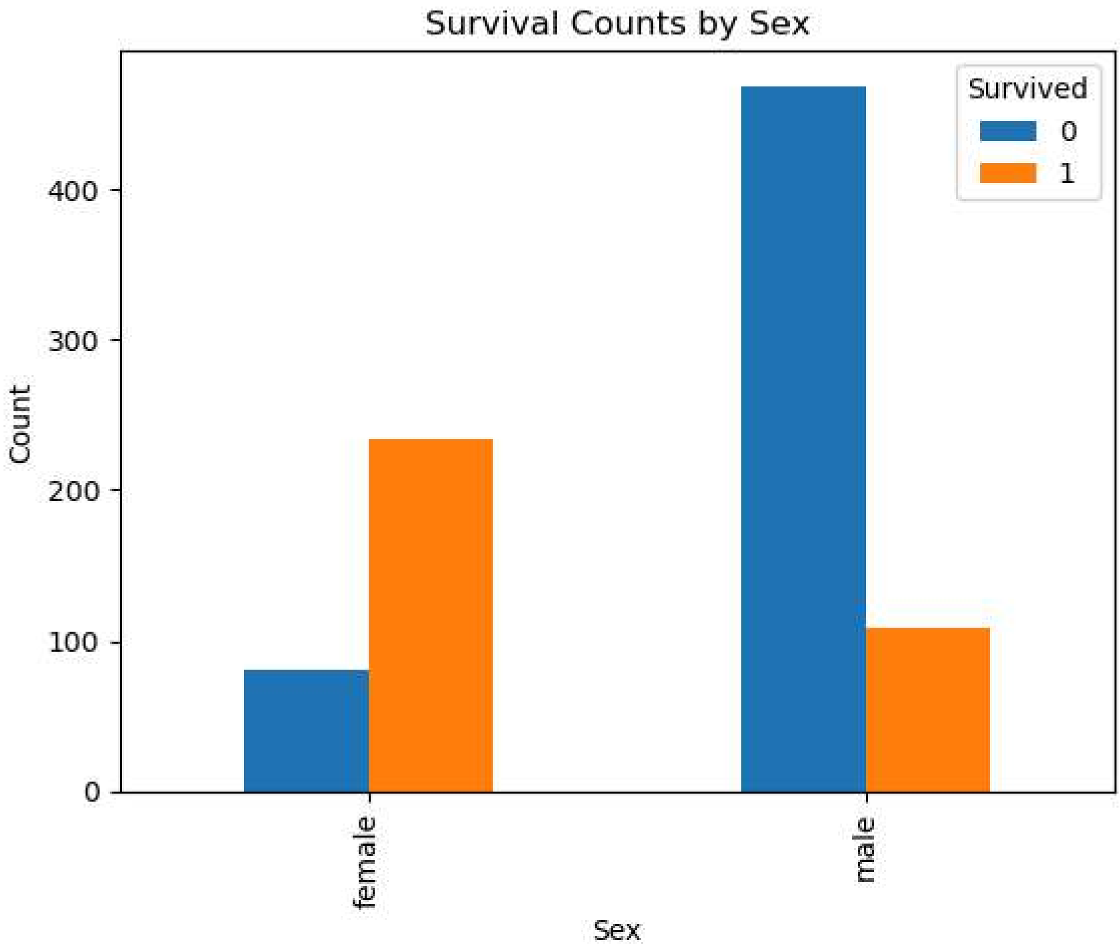


Visualizations

In [60]:

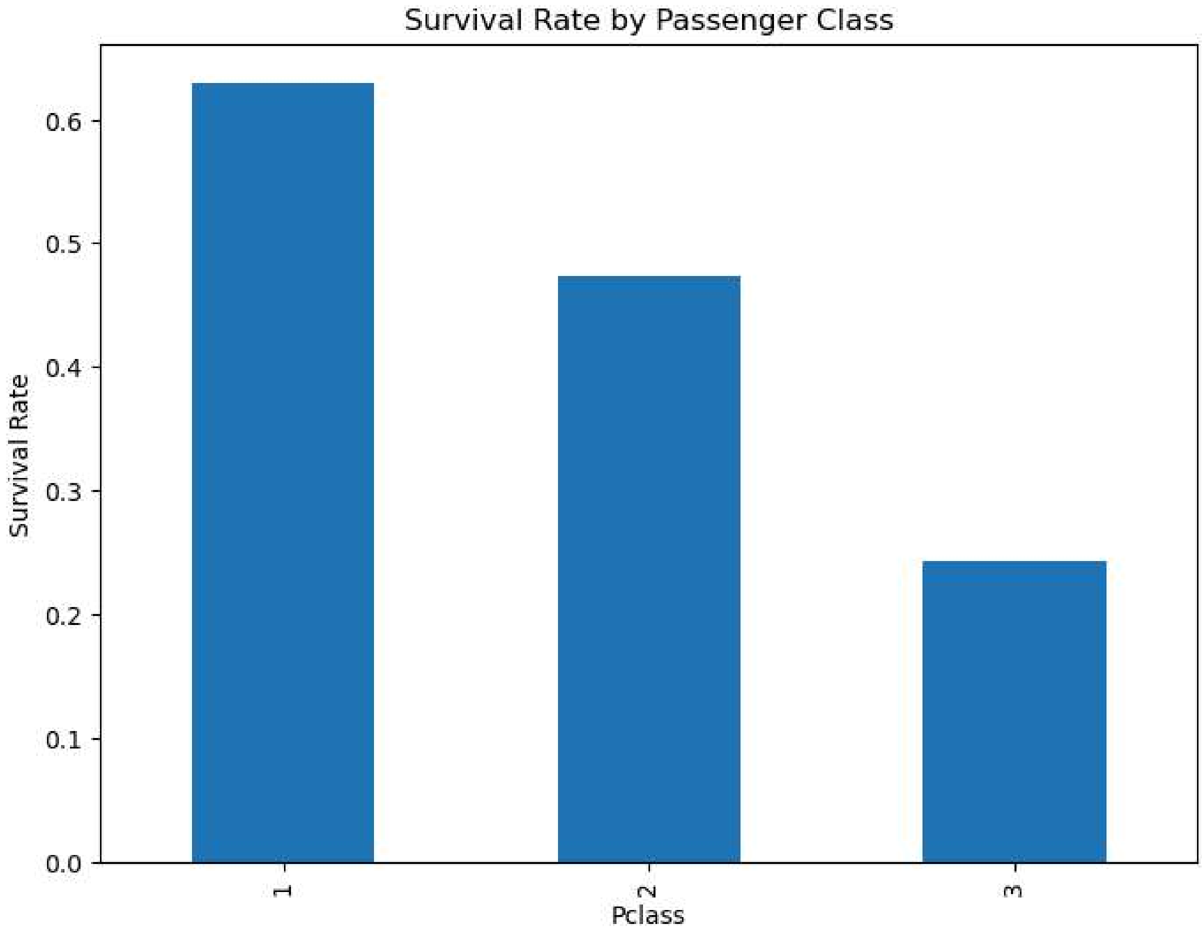
```
plt.figure(figsize=(8,6))
df.groupby(['Sex','Survived']).size().unstack().plot(kind='bar')
plt.title('Survival Counts by Sex')
plt.ylabel('Count')
plt.show()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2680\3043230054.py:2: FutureWarning: The de
fault of observed=False is deprecated and will be changed to True in a future versio
n of pandas. Pass observed=False to retain current behavior or observed=True to adop
t the future default and silence this warning.
  df.groupby(['Sex','Survived']).size().unstack().plot(kind='bar')
<Figure size 800x600 with 0 Axes>
```



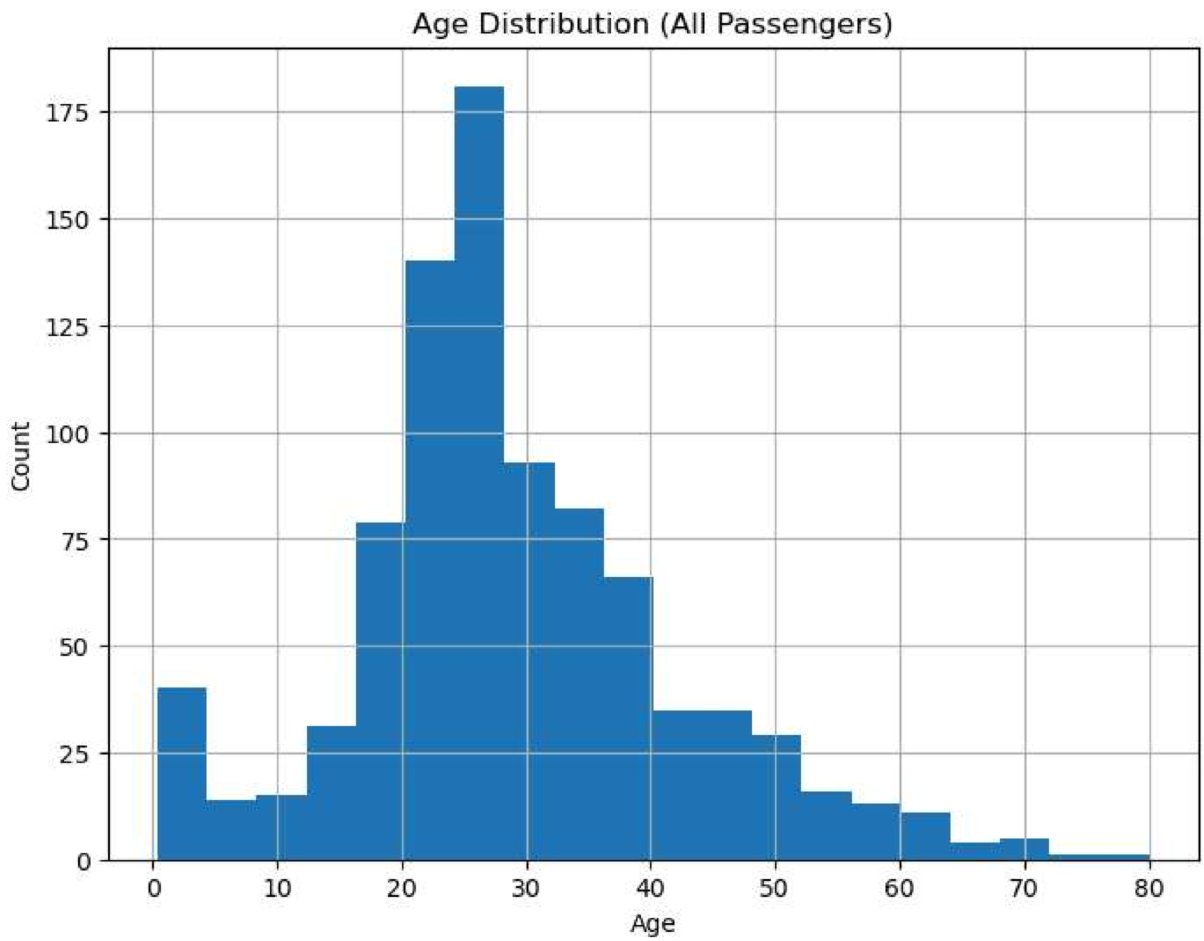
In [62]:
```
plt.figure(figsize=(8,6))
df.groupby('Pclass')['Survived'].mean().plot(kind='bar')
plt.title('Survival Rate by Passenger Class')
plt.ylabel('Survival Rate')
plt.show()
```

```
C:\Users\HP\AppData\Local\Temp\ipykernel_2680\2341338301.py:2: FutureWarning: The de
fault of observed=False is deprecated and will be changed to True in a future versio
n of pandas. Pass observed=False to retain current behavior or observed=True to adop
t the future default and silence this warning.
  df.groupby('Pclass')['Survived'].mean().plot(kind='bar')
```

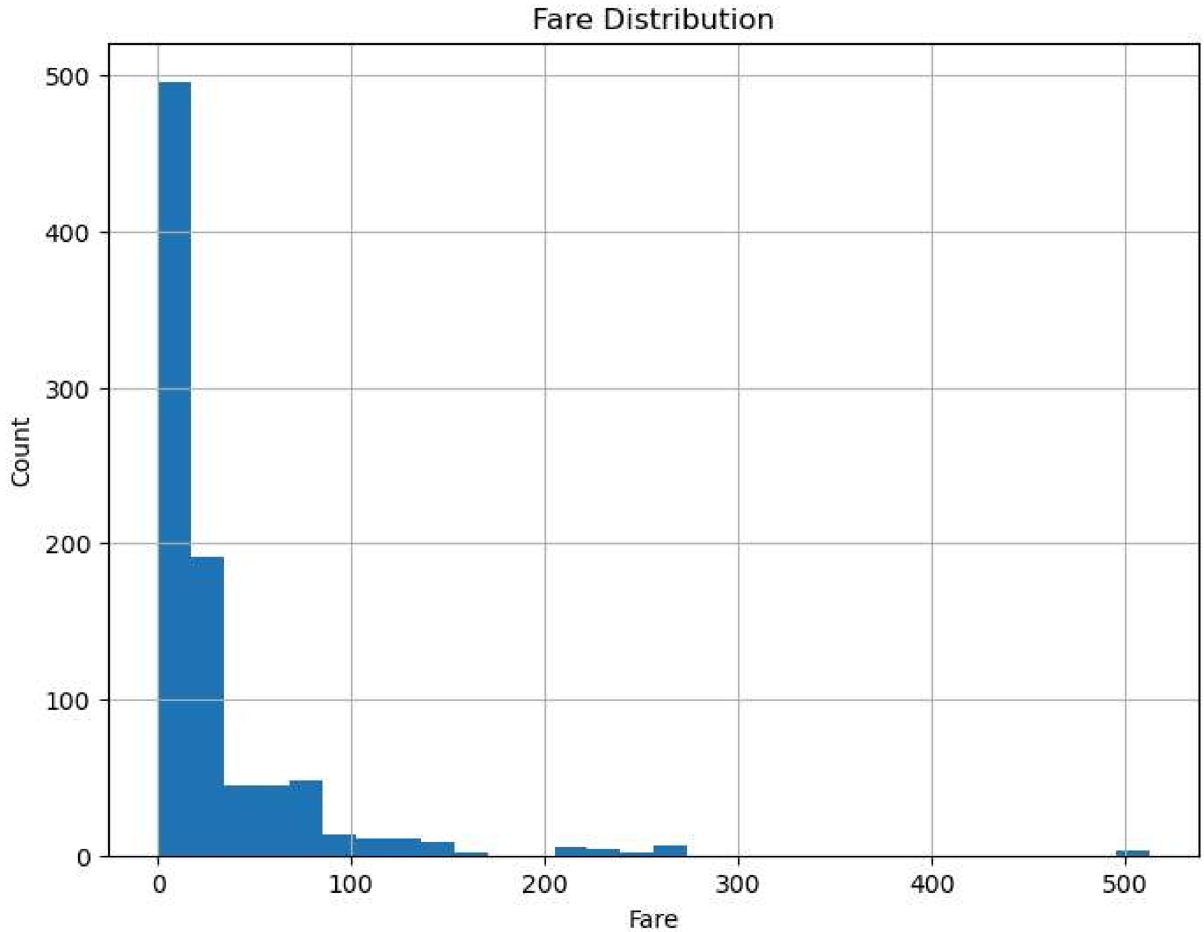


In [64]:

```
plt.figure(figsize=(8,6))
df['Age'].hist(bins=20)
plt.title('Age Distribution (All Passengers)')
plt.xlabel('Age')
plt.ylabel('Count')
plt.show()
```



In [66]:
```
plt.figure(figsize=(8,6))
df['Fare'].hist(bins=30)
plt.title('Fare Distribution')
plt.xlabel('Fare')
plt.ylabel('Count')
plt.show()
```



save cleaned dataset

In [69]:
```
df.to_csv('titanic_cleaned.csv', index=False)
print("Cleaned dataset saved as titanic_cleaned.csv")
```

```
Cleaned dataset saved as titanic_cleaned.csv
```

In [ ]:
```
!jupyter nbconvert --to html "Untitled37.ipynb" --output-dir "C:/Users/HP/Desktop/d
```

In [ ]:
```

```